

회원 관리 프로그램 — 과제 목적 정의서

항목	내용
문서명	회원 관리 프로그램 과제 목적 정의서
문서 유형	과제 목적 (Objective / Background)
버전	v1.0
작성일	2026-06-03
작성 도구 언어	Python
대상	교육과정 수강생
상태	확정(Baseline)

이 문서를 읽는 분께(안내) 이 문서는 "왜 이 과제를 하는가"를 설명합니다. 코드를 어떻게 짜는지(=How)는 뒤의 **구현요구사항**, **화면정의서**, **기능정의서**, **PRD**, **TRD** 문서에서 다룹니다. 이 문서는 "무엇을 배우고, 그것이 실제 회사에서 어떻게 쓰이는가(=Why)"에만 집중합니다.

1. 과제 개요

수업 시간에 배운 **파이썬(Python)** 언어를 사용하여 **회원 관리 프로그램(콘솔 기반)** 을 직접 만든다. 이 과제의 목적은 다음 한 문장으로 요약된다.

"프로그램이 데이터를 만들고(Create), 읽고(Read), 고치고(Update), 지우는>Delete) 기본 동작 원리를 직접 구현해 보고, 그 데이터를 파일에 저장·복원하는 흐름을 이해한다."

원본 과제 지시서에 명시된 목적은 다음과 같다.

- 수업에서 배운 파이썬 언어를 사용한다.
- 과제 수행을 통해 **프로그램 동작 원리를 이해한다**.
- 배운 내용을 **응용하는 능력**을 키운다.

2. 무엇을 배우는가 (핵심 학습 목표)

이 과제 하나로 익히게 되는 4가지 핵심 개념이다. 각 개념은 **실무에서 매일 쓰이는 기술**이다.

2-1. CRUD (데이터 처리의 4대 기본 동작)

메뉴	영어 용어	실무에서 같은 일
회원 추가	Create	쇼핑몰 회원가입, 게시글 작성, 상품 등록
회원 목록 보기	Read	내 주문 내역 조회, 친구 목록 보기
회원 정보 수정	Update	배송지 변경, 비밀번호 변경, 프로필 수정
회원 삭제	Delete	회원 탈퇴, 장바구니 상품 빼기

비유로 이해하기 CRUD는 "수첩 사용법"과 똑같다. 수첩에 **새 연락처를 적고(C)**, **펼쳐서 읽고(R)**, **번호가 바뀌면 고쳐 쓰고(U)**, **필요 없어지면 지운다(D)**. 세상의 거의 모든 앱(인스타그램, 배달앱, 은행앱)은 이 4가지 동작의 조합으로 만들어진다.

2-2. 딕셔너리(Dictionary) — 데이터를 "이름표"로 묶기

한 명의 회원은 이름·전화번호·주소·종류 같은 여러 정보를 가진다. 이 정보들을 따로따로 변수로 두면 관리가 어렵다. 딕셔너리는 이를 **이름표(key)**와 **값(value)** 한 쌍으로 묶어준다.

```
# 예시: 회원 한 명 = 딕셔너리 한 개
회원 = {"이름": "윤아", "전화번호": "010-1234-5678", "주소": "서울시", "종류": "친구"}
print(회원["이름"]) # 출력: 윤아
```

실무 연결: 웹/앱이 서버와 주고받는 데이터 형식인 **JSON**은 사실상 딕셔너리와 같은 구조다. 딕셔너리를 이해하면 API 응답 데이터를 다루는 실무 감각으로 바로 이어진다.

2-3. 바이너리 파일 저장 — 프로그램을 꺼도 데이터가 남게 하기

프로그램을 종료하면 메모리(딕셔너리)에 있던 데이터는 사라진다. 다시 켜를 때도 회원 정보가 남아 있으려면 **파일**에 저장해야 한다.

비유로 이해하기 메모리는 "화이트보드"다. 전원을 끄면(=프로그램 종료) 다 지워진다. 파일 저장은 그 내용을 "노트에 옮겨 적는 것"이다. 다음에 다시 화이트보드에 옮겨 적으면(=파일 읽기) 이어서 작업할 수 있다.

실무 연결: 실무에서는 보통 파일 대신 **데이터베이스(DB)**를 쓴다. 하지만 "메모리의 데이터를 영구 저장소에 보존한다"는 개념(영속화, Persistence)은 완전히 동일하다. 이 과제는 DB의 축소판을 파일로 경험하는 것이다.

2-4. 유효성 검사 & 예외 처리 — "잘못된 입력"으로부터 프로그램 지키기

사용자는 언제나 엉뚱한 값을 넣는다(빈 값, 글자 수 초과, 형식이 틀린 전화번호 등). 이를 미리 거르는 것이 **유효성 검사(Validation)**, 그래도 발생한 오류로 프로그램이 죽지 않게 막는 것이 **예외 처리(Exception Handling)** 다.

비유로 이해하기 놀이공원 입구의 키 측정 막대가 **유효성 검사**다. 기준을 통과한 사람만 들여보낸다. 그래도 사고가 나면 작동하는 안전 그물이 **예외 처리**다.

실무 연결: 실제 서비스 장애의 상당수는 "예상치 못한 입력"에서 시작된다. 입력값 검증은 신입 개발자가 가장 먼저, 가장 자주 지적받는 항목이다.

3. 이 과제의 실무적 위치 (왜 회원 관리인가)

"회원 관리"는 가장 단순하면서도 모든 백엔드 시스템의 **원형(原型)**이다.

[이 과제]		[실무 시스템]
회원 디렉터리	→	사용자(User) 테이블
바이너리 파일	→	관계형 데이터베이스(RDB)
콘솔 메뉴 입력	→	REST API 엔드포인트
유효성 검사 함수	→	서버 입력 검증 / 프론트엔드 폼 검증

즉, 이 과제를 완성하면 **"입력 → 검증 → 처리 → 저장 → 조회"** 라는 모든 데이터 시스템의 기본 골격을 손으로 직접 만들어 본 것이 된다.

4. 학습 후 도달 목표 (역량 기준)

이 과제를 완료한 수강생은 다음을 **혼자서** 할 수 있어야 한다.

- 반복되는 콘솔 메뉴(무한 루프 + 메뉴 분기)를 설계할 수 있다.
- 여러 항목을 가진 데이터를 디렉터리로 모델링할 수 있다.
- `pickle` 을 이용해 데이터를 파일에 저장하고 다시 불러올 수 있다.
- 정규식 등으로 입력 형식을 검증하고, `try/except` 로 오류 상황을 안전하게 처리할 수 있다.
- "같은 이름이 여러 명일 때", "없는 회원을 지우려 할 때" 같은 **예외 상황(엡지 케이스)** 을 스스로 떠올리고 대응할 수 있다.

5. 제출 정보 (원본 과제 지시서 기준)

항목	내용
제출 방법	이메일 제출 (<code>kbigdata.edu@gmail.com</code>)
메일 본문 포함 항목	① 소스 코드 복사 ② 결과 화면 캡처 ③ 플로우 차트 ④ 과제 소감(필수)

안내: 본 문서 세트는 "실제 제출용"이 아니라 **교육과정용 기획·요구 문서**로 작성되었다. 실제 수업에서 사용할 경우 제출처·마감일 등 제출 정보를 현행화해야 한다.

6. 평가 관점 (무엇을 잘하면 잘한 것인가)

평가 영역	잘된 예	흔한 실수
기능 완성도	5개 메뉴가 모두 동작	수정/삭제에서 동명이인 처리 누락
데이터 영속화	종료 후 재실행 시 데이터 유지	저장만 하고 시작 시 읽기 누락
유효성 검사	잘못된 전화번호 거부	어떤 값이든 그냥 저장
예외 처리	잘못 입력해도 프로그램이 안 죽음	오류 시 프로그램 강제 종료
코드 가독성	기능별 함수 분리, 의미 있는 변수명	모든 코드가 한 덩어리

7. 후속 문서 안내

문서	다루는 내용
구현요구사항 정의서	각 기능의 입·출력, 유효성 규칙, 예외 상황을 빠짐없이 명세
화면 정의서	콘솔 화면(메뉴·입력·출력)별 레이아웃, 화면 전이, 공통 메시지
기능 정의서	함수 단위 기능 명세(입력·처리 로직·출력), 함수 호출 관계
PRD (제품 요구사항)	사용자 관점의 기능 요구사항, 전체 사용자 흐름, 인수 기준
TRD (기술 요구사항)	시스템 아키텍처, 모듈/함수 설계, 데이터 구조, 기술 스택

문서 읽는 순서 권장: 과제목적 → 구현요구사항 → 화면정의서 → 기능정의서 → PRD → TRD